

CS257 Linear and Convex Optimization

Homework 6

Due: November 2, 2020

October 26, 2020

For this assignment, you should submit a pdf file as well as your source code (.py files). The pdf file should include all necessary figures, the outputs of your Python code, and your answers to the questions. Do NOT write your answers in any of the .py files. Please TYPE your answers. You may use Latex, Word, or any other software you prefer.

In this assignment, you will implement gradient descent with constant step size. First complete the function `gd_const_ss` in `gd.py`.

1. Consider the following quadratic program,

$$\min_{\mathbf{x} \in \mathbb{R}^2} f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{Q} \mathbf{x} \quad (1)$$

where

$$\mathbf{Q} = \begin{pmatrix} 1 & 0 \\ 0 & \gamma \end{pmatrix}.$$

with $\gamma > 0$. In terms of coordinates,

$$f(x_1, x_2) = \frac{1}{2}x_1^2 + \frac{\gamma}{2}x_2^2.$$

We know the optimal solution is $\mathbf{x}^* = \mathbf{0}$ and $f^* = f(\mathbf{x}^*) = 0$.

- (a). What is the smallest L such that f is L -smooth?
- (b). Suppose $\gamma = 10$. Complete `p1.py` and run the gradient descent algorithm you implemented with step sizes 0.22, 0.1, 0.01, 0.001. (Note: for step size 0.22, you may want to limit the maximum number of iterations to a small number, say 10.) Does it converge in each case? When it does converge, how many iterations does it take? In each case, plot the 2D trajectory of the sequence $\mathbf{x}_k$ and the function values $f(\mathbf{x}_k)$. You can use the function provided in `utils.py` or write your own code. Note that $f(\mathbf{x}_k)$ is also the approximation error since $f(\mathbf{x}^*) = 0$.
- (c). For $\gamma = 1, 0.1, 0.01, 0.001$, run gradient descent with step size 1. How many iterations do you need in each case? How does the number of iterations change as γ decreases?

Solution. One possible implementation of `gd_const_ss` is given below.

```
def gd_const_ss(fp, x0, stepsize, tol=1e-5, maxiter=100000):  
    x_traces = [np.array(x0)]
```

```

x = np.array(x0)

for k in range(maxiter):
    grad = fp(x)
    if np.linalg.norm(grad) < tol:
        break

    x -= stepsize * grad
    x_traces.append(np.array(x))

return x_traces

```

- (a). The eigenvalues of $\mathbf{Q}$ are 1 and γ , so $L = \max\{1, \gamma\}$.
- (b). The algorithm diverges for step size 0.22 and converges in all other cases. The number of iterations is shown in Table 1. The plots are shown in Figures 1 - 4. When step size is 0.22, $f(\mathbf{x}_k)$ grows exponentially with k . In all other cases, $f(\mathbf{x}_k)$ decays exponentially.

step size	number of iterations
0.1	110
0.01	1146
0.001	11508

Table 1: $\gamma = 10$

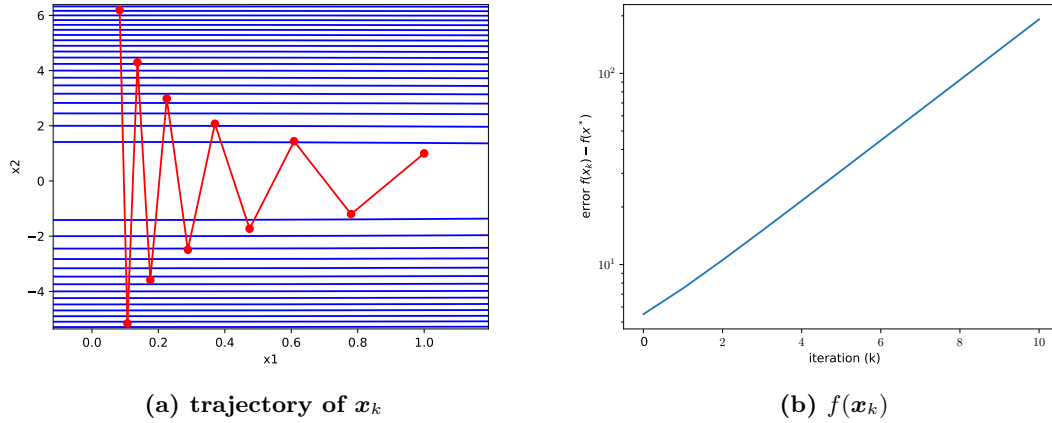
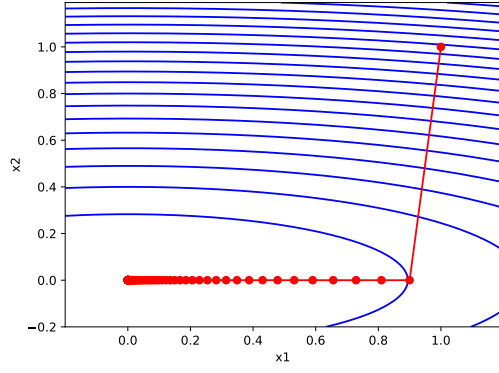
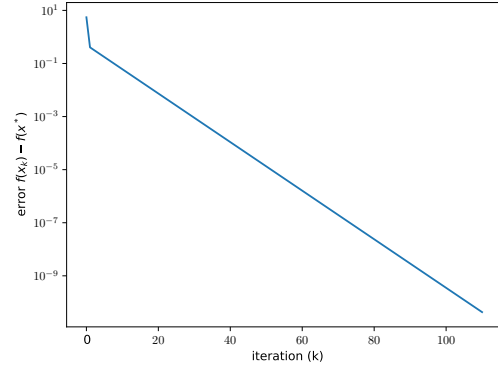


Figure 1: $\gamma = 10$, set size 0.22

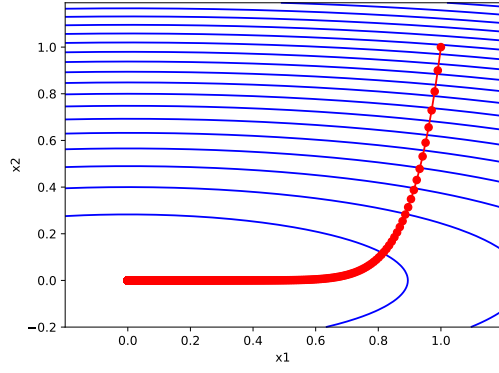


(a) trajectory of x_k

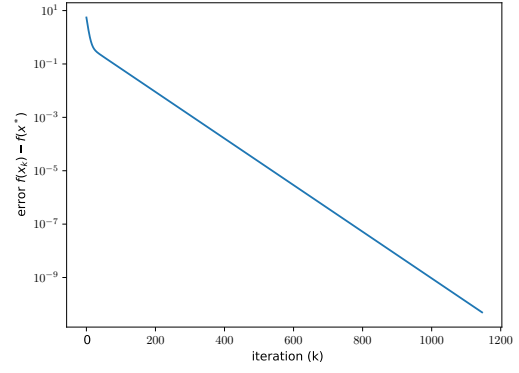


(b) $f(x_k)$

Figure 2: $\gamma = 10$, set size 0.1

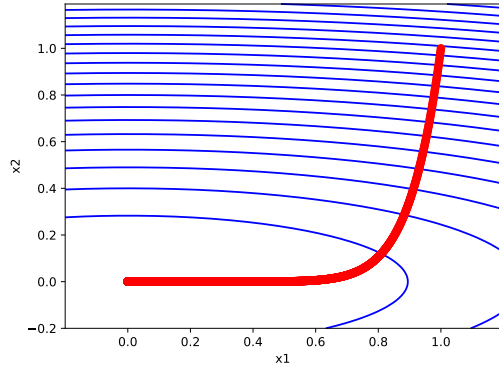


(a) trajectory of x_k

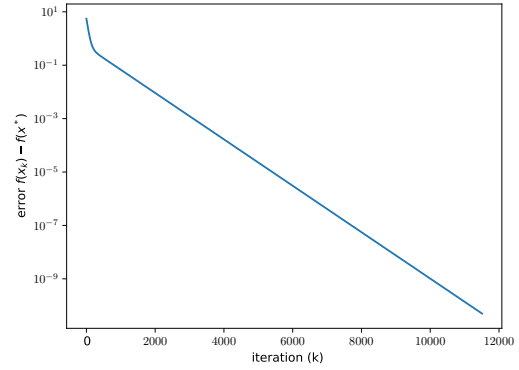


(b) $f(x_k)$

Figure 3: $\gamma = 10$, set size 0.01



(a) trajectory of x_k



(b) $f(x_k)$

Figure 4: $\gamma = 10$, set size 0.001

- (c). See Table 2. The number of iterations increases very fast as γ decreases. It is roughly inversely proportional to γ .

γ	number of iterations
1	1
0.1	88
0.01	688
0.001	4603

Table 2: step size = 1

□

2. Consider the least squares problem in Problem 3(a) of Homework 5, i.e.

$$\min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$$

where

$$\mathbf{X} = \begin{pmatrix} 2 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 3 \\ 2 \\ 2 \end{pmatrix}.$$

Use your implementation of gradient descent to solve this least squares problem. You may follow provided codes of problem 1. Use any stepsize you find appropriate. Compare your solution with the solution you found in HW5, and the solution you find by solving the normal equation using `np.linalg.solve`.

Solution. The objective function is

$$f(\mathbf{w}) = \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w} - 2\mathbf{y}^T \mathbf{X} \mathbf{w} + \|\mathbf{y}\|^2$$

with gradient

$$\nabla f(\mathbf{w}) = 2\mathbf{X}^T \mathbf{X} \mathbf{w} - 2\mathbf{X}^T \mathbf{y}$$

Since

$$\mathbf{X}^T \mathbf{X} = \text{diag}\{4, 1\}$$

f is 8-smooth.

One possible implementation is given below with step size equal to $0.1 < 1/8$. Any step size in $(0, 1/4)$ will work (we only proved convergence for $t \in (0, 1/L]$, but it turns out any $t \in (0, 2/L)$ works).

The output is given below.

```
number of iterations: 58
solution: [1.5      1.99999521]
value: 4.0000000000229345
closed-form solution: [1.5 2. ]
```

The solution found in HW5 is `[1.5 2. ]`. All three solutions are practically the same.

□

3. Logistic regression is a statistical model that uses a logistic function to analyze the relationship between a binary dependent variable and one or more dependent variables. You are required to build a logistic-regression classifier for a toy dataset. Each sample in the dataset includes a 2-dimension vector that represents the feature of the sample and a binary label that identifies the type of the sample. The model takes the feature as input and predicts the type, and it can be expressed as:

$$\sigma_{\theta}(\mathbf{x}) = \frac{1}{1 + e^{-\theta^T \mathbf{x}}}$$

where θ represents the parameters of the model and $\mathbf{x}$ is the input. Here we add a bias term to the model by having one more dimension for both θ and $\mathbf{x}$ with a 1 appended to $\mathbf{x}$ (see the provided code), so feature vector of each data sample actually has 3 dimensions. If we take $\sigma_{\theta}(\mathbf{x})$ as the probability that the sample belongs to type 1, then $1 - \sigma_{\theta}(\mathbf{x})$ is the probability the sample belongs to type 0. The predicted label is determined by the one with the greater value, i.e. if $\sigma_{\theta}(\mathbf{x}) > 1 - \sigma_{\theta}(\mathbf{x})$ then the predicted type is 1.

The objective function of this problem is given below:

$$f(\theta) = - \sum_{i=1}^N [y^i \log \sigma_{\theta}(\mathbf{x}^i) + (1 - y^i) \log(1 - \sigma_{\theta}(\mathbf{x}^i))],$$

where N is the total number of samples in the dataset, $\mathbf{x}^i$ and y^i are the feature and the label of the i^{th} sample respectively. We want to minimize this objective function:

$$\min_{\theta} f(\theta).$$

- (a). Prove that the objective function is convex. Hint: Use the convexity of known functions and the convexity-perserving operations.
- (b). Prove that the gradient of the objective function is

$$\sum_{i=1}^N (\sigma_{\theta}(\mathbf{x}^i) - y^i) \mathbf{x}^i.$$

Hint: $g(x) = \frac{1}{1+e^{-x}}$, $\nabla g(x) = g(x)(1 - g(x))$.

- (c). The toy dataset is given in **p3.py**. Optimize your model (using **gradient descent**) and test it on the dataset. Follow the guidance in **p3.py** and complete it. Use a **screenshot** to display the accuracy (the ratio of samples whose labels are predicted correctly by your model).

Hint: Use matrix operations to calculate the function values and gradients. Some numpy functions for matrix operations that might be useful are listed below. Refer to online documentation for their usage. `numpy.matmul()`, `numpy.multiply()`, `numpy.exp()`, `numpy.reshape()`.

Proof. (a). Objective function:

$$f(\theta) = \sum_{i=1}^N [y^i \log(1 + e^{-\theta^T \mathbf{x}^i}) + (1 - y^i)(\theta^T \mathbf{x}^i + \log(1 + e^{-\theta^T \mathbf{x}^i}))] \quad (2)$$

We know $\log(1 + e^x)$ is convex and nondecreasing, $-\theta^T \mathbf{x}$ is convex, thus $\log(1 + e^{-\theta^T \mathbf{x}})$ is convex. Since $\log(1 + e^{-\theta^T \mathbf{x}})$ is convex and $\theta^T \mathbf{x}$ is convex, $\theta^T \mathbf{x} + \log(1 + e^{-\theta^T \mathbf{x}})$ is convex. As y^i can only be

0 or 1, $f(\boldsymbol{\theta})$ is convex.

(b). Let $g(\mathbf{x}) = \frac{1}{1+e^{-\mathbf{x}}}$,

$$\begin{aligned}
\frac{\partial f(\boldsymbol{\theta})}{\partial \theta_j} &= - \sum_{i=1}^N \left[y^i \frac{1}{\sigma_{\boldsymbol{\theta}}(\mathbf{x}^i)} \frac{\partial \sigma_{\boldsymbol{\theta}}(\mathbf{x}^i)}{\partial \theta_j} - (1 - y^i) \frac{1}{1 - \sigma_{\boldsymbol{\theta}}(\mathbf{x}^i)} \frac{\partial \sigma_{\boldsymbol{\theta}}(\mathbf{x}^i)}{\partial \theta_j} \right] \\
&= - \sum_{i=1}^N \left[y^i \frac{1}{g(\boldsymbol{\theta}^T \mathbf{x}^i)} - (1 - y^i) \frac{1}{1 - g(\boldsymbol{\theta}^T \mathbf{x}^i)} \right] \frac{\partial g(\boldsymbol{\theta}^T \mathbf{x}^i)}{\partial \theta_j} \\
&= - \sum_{i=1}^N \left[y^i \frac{1}{g(\boldsymbol{\theta}^T \mathbf{x}^i)} - (1 - y^i) \frac{1}{1 - g(\boldsymbol{\theta}^T \mathbf{x}^i)} \right] g(\boldsymbol{\theta}^T \mathbf{x}^i) (1 - g(\boldsymbol{\theta}^T \mathbf{x}^i)) x_j^i \\
&= - \sum_{i=1}^N [y^i (1 - g(\boldsymbol{\theta}^T \mathbf{x}^i)) - (1 - y^i) g(\boldsymbol{\theta}^T \mathbf{x}^i)] x_j^i \\
&= \sum_{i=1}^N [\sigma_{\boldsymbol{\theta}}(\mathbf{x}^i) - y^i] x_j^i
\end{aligned} \tag{3}$$

Therefore, the gradient is:

$$\sum_{i=1}^N (\sigma_{\boldsymbol{\theta}}(\mathbf{x}^i) - y^i) \mathbf{x}^i$$

□